

Auditoría Cruzada de Calidad: Devs vs QA

Actividad Práctica - Aseguramiento de la Calidad del Software
NRC 30733

Objetivo General

Simular un entorno corporativo real donde coexisten **equipos de desarrollo** y **equipos de aseguramiento de calidad**, generando una dinámica de tensión constructiva: unos construyen (con vicios intencionales) y otros auditan, priorizan y exigen mejoras bajo el estándar **ISO/IEC 25010**.

1. Distribución de los 7 Grupos

Rol	Grupos	Responsabilidad
✂ Squads de Desarrollo (Devs)	Grupos 1, 2 y 3	Construyen un proyecto “Legacy” con problemas intencionales inyectados y lo entregan antes de la clase .
Q Squads de QA (Auditores)	Grupos 4, 5, 6 y 7	Auditan los proyectos, extraen métricas, priorizan deuda técnica y presentan hallazgos ejecutivos.

Asignación de Proyectos a los QA

- **Grupo 4** audita el proyecto del **Grupo 1**.
- **Grupo 5** audita el proyecto del **Grupo 2**.
- **Grupo 6** audita el proyecto del **Grupo 3**.
- **Grupo 7** actúa como “**Arquitecto de Calidad**”: no audita un solo proyecto, sino que hace un **análisis comparativo (benchmarking)** de los 3 dashboards de SonarCloud para determinar cuál proyecto está en mejor/peor estado relativo y por qué.

Trabajo Previo a la Clase

Cada grupo Dev debe entregar **durante la clase** de la clase un repositorio en GitHub con las siguientes características **obligatorias**:

Checklist de “Vicios” a Inyectar

Categoría	Requisito Mínimo
🐛 Bugs / Confiabilidad	Al menos 3 bugs funcionales (ej. división por cero, manejo incorrecto de nulls, bucles infinitos).
⚠️ Code Smells / Mantenibilidad	Al menos 2 funciones con complejidad ciclomática >20, 15 %+ de código duplicado, métodos de más de 50 líneas.
🔧 Ausencia de Pruebas	Cobertura general <30 %, y 0 % de cobertura en módulos críticos (autenticación, pagos o base de datos).
🔒 Vulnerabilidades (Security)	Al menos 2 <i>Hotspots</i> : credenciales hardcodeadas, SQL sin parametrizar, o uso de <code>eval()</code> / <code>exec()</code> .
📁 Dominio del Proyecto	Tamaño mínimo: 5-8 archivos de código, 1 módulo crítico claramente identificable (ej. “Procesador de Pagos”).
Documentación	README con instrucciones de ejecución y justificación “ficticia” de por qué el código está así (presión de tiempo, deuda heredada, etc.).

Sugerencia de proyectos: API REST de e-commerce, sistema de reservas, gestor de tareas con autenticación, o chatbot simple.

2. Cronograma de la Actividad

Fase 1: El Briefing Corporativo

- El docente (como *CTO*) presenta el escenario: “*Adquirimos 3 startups. Cada una dejó un sistema. Su salud es desconocida.*”
- Cada **grupo Dev** tiene **2 minutos** para hacer un *pitch de venta* de su proyecto al CTO y a los QA, defendiendo por qué su código “está bien así” (presión de tiempo, legacy, etc.).
- Se asignan oficialmente los proyectos a los grupos QA.

Fase 2: Minería de Métricas

- Los **4 grupos QA** acceden a los dashboards de SonarCloud y repositorios asignados.
- Extraen métricas bajo la **ISO 25010**: Confiabilidad, Mantenibilidad, Seguridad, Cobertura, Complejidad.
- El **Grupo 7 (Arquitecto)** abre los 3 dashboards simultáneamente y construye una **tabla comparativa** con las métricas clave de los 3 proyectos.
- Los **grupos Dev** permanecen en sus puestos como “*Dev Team on-call*”, disponibles para responder preguntas técnicas de los QA (simulando una sesión de *pair auditing*).

Fase 3: Diagnóstico y Triage

- Los **grupos QA** cruzan métricas con **riesgo de negocio** y crean entre 3 y 5 *Issues* en GitHub del proyecto auditado, redactados como historias de deuda técnica con criterios de aceptación.
- El **Grupo 7** redacta un “**Reporte de Benchmarking**”: ¿Qué proyecto recomendaría adquirir sin refactorizar? ¿Cuál es una bomba de tiempo? ¿Qué patrón de mala práctica se repite en los 3?
- Los **grupos Dev** revisan los *Issues* que los QA están creando y preparan una **defensa técnica** (¿realmente es un bug? ¿es prioritario?).

Fase 4: “Shark Tank” de Calidad — Pitch y Defensa

Dinámica tipo debate por proyecto (8 min por proyecto + 6 min del Grupo 7):

1. **Grupo QA presenta (3 min)**: Semáforo de salud, hallazgo crítico, propuesta priorizada.
2. **Grupo Dev defiende (2 min)**: Acepta, rechaza o negocia los *Issues* planteados (simulando negociación real de Sprint).
3. **Preguntas del CTO y la clase (3 min)**: El docente y otros grupos hacen preguntas cruzadas.

Al final, el **Grupo 7 (Arquitecto)** presenta su **benchmarking comparativo (3 min)**: “*El proyecto 2 es el más riesgoso porque, aunque tiene menos bugs, su módulo de pagos tiene 0% de cobertura y alta complejidad...*”.

Fase 5: Retroalimentación y Cierre

Preguntas de Reflexión:

- ¿Fue más difícil **crear código malo** o **detectarlo**? (Reflexión sobre la naturaleza humana del desarrollo).
- ¿Cómo se sintió la tensión Dev vs QA? ¿Fue colaborativa o adversarial?
- ¿Cómo integrar **Quality Gates** en CI/CD para que los vicios inyectados hoy **nunca lleguen a producción**?

3. Rúbricas de Evaluación Diferenciadas

Para los Grupos Devs (1, 2 y 3)

Criterio		Excelente (100 %)	Aceptable (70 %)	Deficiente (40 %)
Calidad del “Código Malo”		Inyecta vicios sutiles y realistas que realmente engañan a un revisor rápido.	Cumple el checklist pero los errores son muy obvios.	No cumple el checklist o el proyecto no ejecuta.
Defensa Técnica (Pitch)		Argumenta con criterios de negocio reales (time-to-market, ROI).	Se limita a decir “no hubo tiempo”.	No defiende su proyecto.
Negociación de Issues		Negocia con argumentos técnicos válidos, acepta lo innegable.	Acepta todo sin discusión o rechaza todo sin argumentos.	No participa en la negociación.

Para los Grupos QA (4, 5, 6)

Criterio		Excelente (100 %)	Aceptable (70 %)	Deficiente (40 %)
Interpretación de Métricas		Relaciona métricas con ISO 25010 y riesgo de negocio.	Lee números sin contexto.	Confunde conceptos.
Calidad de los Issues		Tickets con contexto, ubicación, impacto y criterio de aceptación.	Tickets vagos.	No crea tickets.
Pitch Ejecutivo		Traduce hallazgos técnicos a impacto de negocio.	Exceso de jerga técnica.	No sintetiza.

Para el Grupo 7 (Arquitecto de Calidad)

Criterio	Excelente (100 %)	Acceptable (70 %)	Deficiente (40 %)
Análisis Comparativo	Identifica patrones transversales y establece ranking justificado.	Compara números sin análisis de causa raíz.	No hace comparación.
Recomendación Estratégica	Recomienda acciones a nivel de portafolio (qué proyecto refactorizar primero).	Solo describe, no recomienda.	No presenta conclusiones.

¡Éxito en la actividad!